ASSIGNMENT- 08

Bachelor of Computer Applications (Hons. AI & ML)
Semester- III

Subject: Database Management System

Topic: Basic SQL Queries and Aggregate Functions

By
Name: MERIN THOMAS
Reg. No.: 2511021240062

Faculty Name: RENITA JOY
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
August 2026

1. Aim

To perform SQL data retrieval operations using `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`, aggregate functions, and joins on the Paranormal Investigation Agency Management System database.

2. Introduction

SQL queries are used to retrieve, filter, organize, and analyze data stored in relational databases. In the Paranormal Investigation Agency Management System, SQL queries help retrieve client details, track investigation cases, identify assigned investigators, review evidence, generate reports, and check payment statuses.

This assignment demonstrates different SQL queries using the tables created in previous assignments.

3. SQL Queries and Their Outputs

3.1 Retrieve All Client Details

```
SELECT * FROM Clients;
```

Explanation: Displays all records and columns from the Clients table.

Expected output: Five client records, if the sample data from Assignment 7 has been inserted successfully.

3.2 Retrieve Cases with High Urgency

```
SELECT Case_id, Client_id, Case_status, Urgency_level

FROM Cases

WHERE Urgency_level = 'High';
```

Explanation: Displays only cases classified as high urgency.

Expected output: Cases 101 and 105.

3.3 Display Completed Cases

```
SELECT Case_id, Client_id, Investigation_date, Case_status

FROM Cases
```

```
WHERE Case_status = 'Completed';
```

Explanation: Retrieves cases whose investigation status is completed.

Expected output: Cases 101 and 104.

3.4 Sort Cases by Investigation Date

```
SELECT Case_id, Investigation_date, Urgency_level
FROM Cases
ORDER BY Investigation_date ASC;
```

Explanation: Displays cases in ascending order of investigation date, from earliest to latest.

3.5 Count the Total Number of Cases

```
SELECT COUNT(*) AS Total_Cases
FROM Cases;
```

Explanation: Uses the `COUNT()` aggregate function to calculate the total number of cases.

Expected output:

Total_Cases
5

3.6 Count Cases by Status

```
SELECT Case_status, COUNT(*) AS Total_Cases
FROM Cases
GROUP BY Case_status;
```

Explanation: Groups cases according to their status and counts the cases in each group.

Expected output:

Case_status	Total_Cases
Completed	2
In Progress	2
Open	1

3.7 Find the Total Payment Amount

```
SELECT SUM(Amount) AS Total_Payment_Amount
FROM Payments;
```

Explanation: Uses the `SUM()` function to calculate the total amount recorded in the Payments table.

Expected output: ₹63,000.00

3.8 Display Pending Payments

```
SELECT Payment_id, Case_id, Amount, Payment_mode, Payment_status
FROM Payments
WHERE Payment_status = 'Pending';
```

Explanation: Retrieves payment records that have not yet been marked as paid.

Expected output: Payments associated with Cases 102 and 105.

3.9 Retrieve Cases with Client Names Using INNER JOIN

```
SELECT C.Case_id, CL.name AS Client_Name,
    C.Case_status, C.Urgency_level
```

```
FROM Cases C

INNER JOIN Clients CL

ON C.Client_id = CL.client_id;
```

Explanation: Combines the Cases and Clients tables using their related client IDs. It displays each case with the corresponding client's name.

Expected output: Five rows showing case IDs, client names, case statuses, and urgency levels.

3.10 Display Investigators Assigned to Each Case

```
SELECT CI.Case_id, I.name AS Investigator_Name,

    CI.Role_in_case

FROM Case_Investigator CI

INNER JOIN Investigators I

ON CI.Investigator_id = I.investigation_id

ORDER BY CI.Case_id;
```

Explanation: Joins the Case_Investigator junction table with the Investigators table to identify the investigators assigned to each case and their roles.

Expected output: Nine rows representing the investigator assignments in Assignment 7.

3.11 Retrieve Evidence Details with Case Information

```
SELECT E.Evidence_id, E.Case_id, E.Evidence_type,

    E.Description, I.name AS Recorded_By

FROM Evidence E

INNER JOIN Investigators I

ON E.Recorded_by = I.investigation_id;
```

Explanation: Displays evidence records along with the names of the investigators who recorded them.

Expected output: Six rows showing evidence details and investigator names.

3.12 Display Reports with Case Details

```
SELECT R.Report_id, R.Case_id, I.name AS Prepared_By,
    R.Final_verdict, R.Report_date
FROM Reports R
INNER JOIN Investigators I
ON R.Prepared_by = I.investigation_id;
```

Explanation: Retrieves investigation reports along with the names of the investigators who prepared them.

Expected output: Two report records, if the sample data from Assignment 7 has been inserted successfully.

3.13 Find the Highest and Lowest Payment Amounts

```
SELECT MAX(Amount) AS Highest_Payment,
    MIN(Amount) AS Lowest_Payment
FROM Payments;
```

Explanation: Uses `MAX()` and `MIN()` to find the highest and lowest payment amounts.

Expected output:

Highest_Payment	Lowest_Payment
18000.00	8000.00

3.14 Count Cases for Each Client

```
SELECT CL.client_id, CL.name,
    COUNT(C.Case_id) AS Total_Cases
```

FROM Clients CL

LEFT JOIN Cases C

ON CL.client_id = C.Client_id

GROUP BY CL.client_id, CL.name

ORDER BY CL.client_id;

Explanation: Uses a `LEFT JOIN` to display all clients, including clients who have no cases. `COUNT()` calculates the number of cases associated with each client.

Expected output: Five rows, with one case for each client in the sample dataset.

3.15 Find Cases with Payments Above ₹10,000

SELECT Case_id, Amount, Payment_status

FROM Payments

WHERE Amount > 10000

ORDER BY Amount DESC;

Explanation: Retrieves payment records with amounts greater than ₹10,000 and sorts them from highest to lowest.

Expected output: Payments for Cases 105, 101, and 104.

---

4. SQL Concepts Used

SQL Concept	Purpose
`SELECT`	Retrieves data from tables.
`WHERE`	Filters records according to a condition.

`ORDER BY`	Sorts the query results.
`GROUP BY`	Groups records with common values.
`COUNT()`	Counts records.
`SUM()`	Calculates the total of numeric values.
`MAX()`	Finds the highest value.
`MIN()`	Finds the lowest value.
`INNER JOIN`	Retrieves matching records from related tables.
`LEFT JOIN`	Retrieves all records from the left table and matching records from the right table.

5. Result

The SQL queries were designed to retrieve, filter, sort, and summarize information from the Paranormal Investigation Agency Management System. They demonstrate how aggregate functions and joins can be used to analyze case details, investigator assignments, evidence, reports, and payments.

6. Conclusion

This assignment demonstrates the practical use of SQL queries for database retrieval and analysis. The queries help the agency access important information efficiently, monitor investigation progress, track payments, and retrieve related records from multiple tables. These operations support effective database management and decision-making.